

Schema Driven Forms

Rails and PostgreSQL Design Proposal

STATUS	OWNER	LAST UPDATED
Proposed	To be confirmed	15 September 2026
Authors	To be confirmed	
Reviewers	Engineering and product reviewers to be confirmed	
Related documents	JSON Schema definitions and API contract to be added during implementation	
Scope	Schema driven forms for CompanyGoal, CompanyTask, IndividualEmployeeTask, and Employee relationships	

1 Abstract

This proposal recommends a hybrid persistence model for schema driven forms. Each form definition is expressed as JSON Schema and can be stored in a versioned FormDefinition record. User defined, non-relational values are stored in a JSONB data column. Relationships between Account, CompanyGoal, CompanyTask, IndividualEmployeeTask, and Employee use ordinary Rails associations and PostgreSQL foreign keys.

The frontend continues to render one form from JSON Schema. Relationship fields appear in that schema and in API requests and responses, but Rails removes their values from the JSONB payload before saving. This retains runtime field flexibility while preserving tenant isolation, referential integrity, reverse queries, and familiar Active Record behaviour.

2 Goals and Non Goals

Goals	Non goals
Render CompanyGoal and task forms from JSON Schema	Allow arbitrary Rails model names from client input
Add or change non-relational fields without database migrations	Store a second authoritative copy of relationship IDs in JSONB
Enforce account ownership and relationship integrity on the server	Build a general document database or object graph engine
Return consistent field-level validation errors to the frontend	Define production scale targets before usage data is available

3 Persistence Options

Three persistence strategies were considered. The hybrid option is selected because the form fields are dynamic while the business relationships are known in advance.

Approach	Fields	Relationships	Assessment
Fully relational	Typed columns	Foreign keys and join tables	Strong integrity but every field change needs a migration
Hybrid selected	JSONB data	Foreign keys and join tables	Dynamic fields with reliable Rails associations
Fully document based	JSONB data	IDs inside JSONB	Maximum flexibility but application-owned integrity and query logic

Storage and representation remain separate. The API may return a single JSON document containing attributes and relationships even though PostgreSQL stores those values in different places.

4 Proposed Architecture

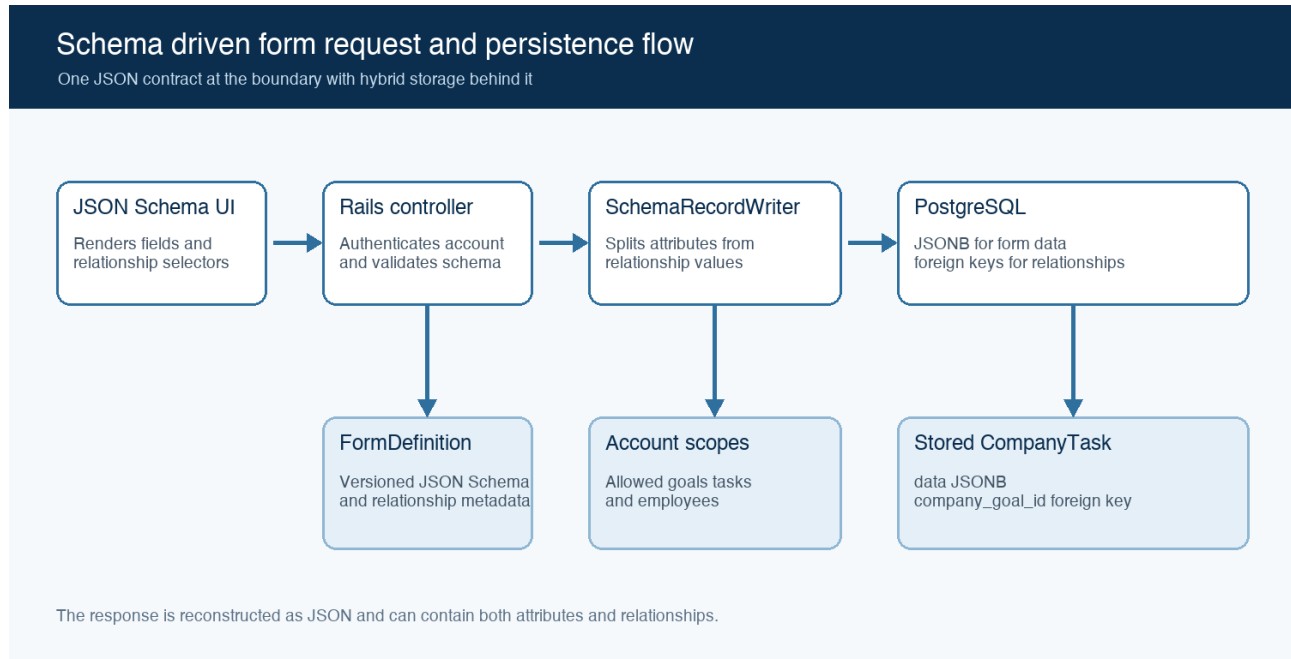


Figure 1 Schema driven form request and persistence flow

Domain relationships

```
Account
├── has many CompanyGoals
├── has many CompanyTasks through CompanyGoals
└── has many Employees

CompanyGoal
├── has many CompanyTasks

CompanyTask
├── has many IndividualEmployeeTasks

IndividualEmployeeTask
├── belongs to CompanyTask
└── belongs to Employee
```

Each employee belongs to one account. A company task inherits its account through its company goal. An individual employee task connects one company task to one employee and stores employee-specific form values in its own JSONB data document.

Database records

Table	Structural columns	JSONB responsibility
form_definitions	id, record_type, version, active	JSON Schema, relationship metadata, and optional UI metadata
company_goals	account_id, form_definition_id	All goal fields including title, status, dates, and priority
company_tasks	company_goal_id, form_definition_id	All company task fields
individual_employee_tasks	company_task_id, employee_id, form_definition_id	Employee-specific status, dates, notes, and other fields
employees	account_id	Existing employee storage remains unchanged

Core components

Component	Responsibility	Failure behaviour
FormDefinition	Stores the versioned schema used to render and validate a record	An invalid or inactive definition cannot be used
Frontend form renderer	Renders ordinary fields and custom relationship selectors	Shows validation or option-loading errors without saving
FormSubmissionValidator	Validates the complete request against JSON Schema	Returns field-level 422 errors
SchemaRecordWriter	Separates JSONB values from allowed relationships	Rejects missing or cross-account targets
Active Record and PostgreSQL	Apply model rules, transactions, foreign keys, and indexes	Roll back the write and return a controlled error

5 Frontend Schema Contract

The schema sent to the frontend describes every form input. Standard JSON Schema keywords define value shape and validation. The application-specific x-relationship annotation tells the form renderer to use a relationship selector. JSON Schema ref remains reserved for schema reuse and does not represent a database foreign key.

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "type": "object",
  "properties": {
    "title": { "type": "string", "minLength": 1 },
    "status": {
      "type": "string",
      "enum": ["not_started", "in_progress", "completed"]
    },
    "due_date": { "type": "string", "format": "date" },
    "company_goal_id": {
      "type": "integer",
      "x-relationship": {
        "association": "company_goal",
        "target": "CompanyGoal",
        "cardinality": "one",
        "optionsUrl": "/company_goals/options"
      }
    }
  }
}
```

```

    },
    "required": ["title", "status", "company_goal_id"],
    "additionalProperties": false
  }
}

```

A separate UI schema may hold widget names and display options if the chosen frontend library supports that convention. The server remains responsible for the relationship allow-list and must not trust model or class names supplied by the browser.

6 Request Lifecycle

1. Rails authenticates the user and resolves `current_account`.
2. The controller selects the active `FormDefinition` for the requested record type. The client need not send `form_definition_id` when only one active definition applies.
3. The frontend submits one document containing ordinary fields and relationship IDs.
4. `FormSubmissionValidator` validates the full document against the selected JSON Schema.
5. `SchemaRecordWriter` reads its server-side relationship allow-list and removes recognised relationship fields from the submitted hash.
6. Each relationship ID is resolved through `current_account`, such as `current_account.company_goals` or `current_account.employees`.
7. The remaining fields are assigned to `record.data` and relationship objects are assigned to Active Record associations.
8. Model validations and PostgreSQL constraints run inside the save transaction.
9. The serializer reconstructs one JSON response containing attributes and relationships.

Create request

```

{
  "company_task": {
    "title": "Contact customers at risk",
    "status": "in_progress",
    "due_date": "2026-10-31",
    "company_goal_id": 10
  }
}

```

After the writer runs, `company_goal_id` is stored in its relational column. The title, status, and due date are stored in `company_tasks.data`. The new company task receives its own ID only after it is saved.

7 Schema Record Writer

The writer is a translation layer rather than a validator for every business rule. It knows which top-level inputs represent relationships, resolves those inputs within the current account, deletes them from the working hash, and assigns the remaining hash to JSONB.

```

class SchemaRecordWriter
  RELATIONSHIPS = {
    "CompanyTask" => {
      "company_goal_id" => {
        association: :company_goal,
        scope: ->(account) { account.company_goals }
      }
    },
    "IndividualEmployeeTask" => {
      "company_task_id" => {
        association: :company_task,
        scope: ->(account) { account.company_tasks }
      },
      "employee_id" => {
        association: :employee,
        scope: ->(account) { account.employees }
      }
    }
  }
}

```

```

}.freeze

def initialize(record, submitted_data, account:)
  @record = record
  @submitted_data = submitted_data.deep_stringify_keys
  @account = account
end

def assign
  relationship_mapping.each do |input_key, definition|
    next unless @submitted_data.key?(input_key)

    relationship_id = @submitted_data.delete(input_key)
    assign_relationship(input_key, relationship_id, definition)
  end

  @record.data = @submitted_data
  @record
end

private

def assign_relationship(input_key, id, definition)
  if id.blank?
    @record.errors.add(input_key, "must be selected")
    return
  end

  relation = definition.fetch(:scope).call(@account)
  target = relation.find_by(id: id)

  unless target
    @record.errors.add(
      input_key,
      "is not available for this account"
    )
    return
  end

  association = definition.fetch(:association)
  @record.public_send("#{association}=", target)
end

def relationship_mapping
  RELATIONSHIPS.fetch(@record.class.name, {})
end
end

```

The mapping is an explicit allow-list. It prevents a client from choosing an arbitrary Ruby class. Assigning the relationship object instead of the raw ID also makes the account scope visible in the implementation.

8 Controller and Error Contract

The controller coordinates authentication, schema selection, submission validation, the writer, persistence, and serialization. In a larger codebase this orchestration can move into a dedicated use-case service, leaving the controller with the same sequence.

```

class CompanyTasksController < ApplicationController
  def create
    definition = FormDefinition.find_by!(
      record_type: "CompanyTask",
      active: true
    )
    submission = params.require(:company_task).to_unsafe_h

    schema_errors = FormSubmissionValidator.new(
      definition,
      submission
    ).errors
    return render_errors(schema_errors) if schema_errors.any?

    task = CompanyTask.new(form_definition: definition)
    SchemaRecordWriter.new(
      task,
      submission,

```

```

    account: current_account
  ).assign

  return render_record_errors(task) if task.errors.any?

  if task.save
    render json: CompanyTaskSerializer.new(task).serializable_hash,
           status: :created
  else
    render_record_errors(task)
  end
end
end
end

```

The example uses `to_unsafe_h` because the submission is dynamic and is never passed into Active Record mass assignment. The selected JSON Schema must reject unexpected keys with `additionalProperties` set to `false`. A production implementation may instead derive a permitted parameter structure from the schema.

Error response

```

{
  "errors": [
    {
      "field": "company_goal_id",
      "message": "is not available for this account"
    }
  ]
}

```

Validation failures return HTTP 422. A cross-account ID receives the same response as a missing ID so the API does not reveal whether another account owns the record. Authentication failures remain 401 and authorisation failures unrelated to field selection remain 403.

9 Validation Responsibilities

Layer	Checks	Purpose
FormDefinition publication	Schema syntax, supported keywords, allowed relationship declarations	Prevents a broken schema from becoming active
Frontend	Required values, types, enum values, dates, and local display rules	Provides immediate feedback only
Backend JSON Schema	The complete submitted document including relationship ID shape	Treats the server as authoritative
SchemaRecordWriter	Relationship existence, account ownership, and assignment eligibility	Enforces tenant and relationship rules
Active Record	Required associations and cross-record business rules	Protects non-controller write paths
PostgreSQL	Not-null constraints, foreign keys, unique indexes, and transaction atomicity	Provides the final integrity boundary

The JSON Schema format keyword may be annotation-only depending on validator configuration. Date and date-time formats must therefore be explicitly enabled in the chosen validator or checked by an application validator before persistence.

Cross account model validation

```

class IndividualEmployeeTask < ApplicationRecord
  belongs_to :company_task
  belongs_to :employee
  belongs_to :form_definition
end

```

```

validate :employee_and_task_share_account

private

def employee_and_task_share_account
  return unless employee && company_task

  task_account_id = company_task.company_goal.account_id
  return if employee.account_id == task_account_id

  errors.add(:employee, "must belong to the task account")
end
end

```

10 Account Scoping and Security

The account scope applies twice. The options endpoint limits what the frontend displays, and the writer applies the same restriction again when saving. The second check is required because a caller can bypass the frontend or alter the submitted ID.

```

def options
  goals = current_account.company_goals
    .where("data->>'status' = ?", "active")
    .order(Arel.sql("data->>'title' ASC"))

  render json: goals.map { |goal|
    { id: goal.id, label: goal.data["title"] }
  }
end

```

- Resolve goals with `current_account.company_goals` rather than `CompanyGoal.find`.
- Resolve employees with `current_account.employees` rather than `Employee.find`.
- Resolve company tasks with `current_account.company_tasks` through company goals.
- Never call `constantize` on a model type supplied by the client.
- Authorise the chosen `FormDefinition` when definitions can vary by account.
- Avoid logging sensitive JSONB values unless the logging policy explicitly permits them.

11 Consistency Querying and Versioning

Create and update behaviour

A complete form submission replaces `record.data`. A partial update must merge submitted non-relational values into the existing document or validate a reconstructed complete document. The API should distinguish PUT-style replacement from PATCH-style merge rather than relying on implicit behaviour.

The JSONB data and relational assignments should be saved in one database transaction. A foreign-key or model-validation failure then leaves neither part partially updated.

Filtering and indexes

```

CompanyTask.where("data->>'status' = ?", "in_progress")

CompanyTask.where(
  "(data->>'due_date')::date <= ?",
  Date.current
)

add_index :company_tasks,
  "(data->>'status')",
  name: "index_company_tasks_on_json_status"

```

Expression indexes should be created only for fields that are used frequently in filters or sorts. Date casts require schema validation to prevent malformed date values from breaking queries or index creation.

Schema versions

A saved record should retain its `form_definition_id` when the exact schema version matters for future display, validation, audit, or migration. Publishing a new definition creates a new version rather than mutating the contract for existing records. If there is one fixed schema per Rails model, the schema may instead live in source control and the client does not need to send `form_definition_id`.

12 Alternatives Considered

Alternative	Reason considered	Reason not selected
All fields and relationships in columns	Native Rails behaviour and strongest SQL ergonomics	Conflicts with runtime-configurable fields and requires frequent migrations
All fields and relationship IDs in JSONB	One self-contained document and no relationship migrations	No ordinary foreign keys, harder reverse queries, and custom integrity logic
Duplicate relationship IDs in columns and JSONB	Raw JSONB appears self-contained	Creates two sources of truth and permanent synchronisation risk
Generic polymorphic relationship table	Supports administrator-defined relationships	Adds complexity that is unnecessary while relationship types remain known

13 Open Questions

- Will each record retain the exact `FormDefinition` version used at creation, or always render with the latest active version?
- Will forms contain nested objects or arrays that require recursively generated permitted parameters?
- Which frontend JSON Schema library is in use, and should relationship controls use custom schema keywords or a separate UI schema?
- What makes an employee assignable beyond belonging to the account?
- Which JSONB fields require production indexes based on expected reports and filters?

14 Decision and Next Steps

Proceed with the hybrid design. Store non-relational form values in JSONB, store known relationships as foreign keys, describe relationship controls in the frontend schema, and reconstruct a unified JSON document at the API boundary. Treat PostgreSQL relationships and the JSONB data document as separate authoritative stores for their respective concerns.

Milestone	Deliverable	Exit criteria
M1	Models, migrations, account associations, and one CompanyTask schema	A task saves JSONB fields and an account-scoped goal relationship
M2	FormSubmissionValidator, SchemaRecordWriter, controllers, and error contract	Automated tests cover valid, invalid, missing, and cross-account relationships
M3	Frontend relationship widget and scoped options endpoints	The form renders, submits, and displays field-level errors end to end
M4	Index review, schema version policy, audit logging, and rollout	Query plans are acceptable and schema changes are recoverable